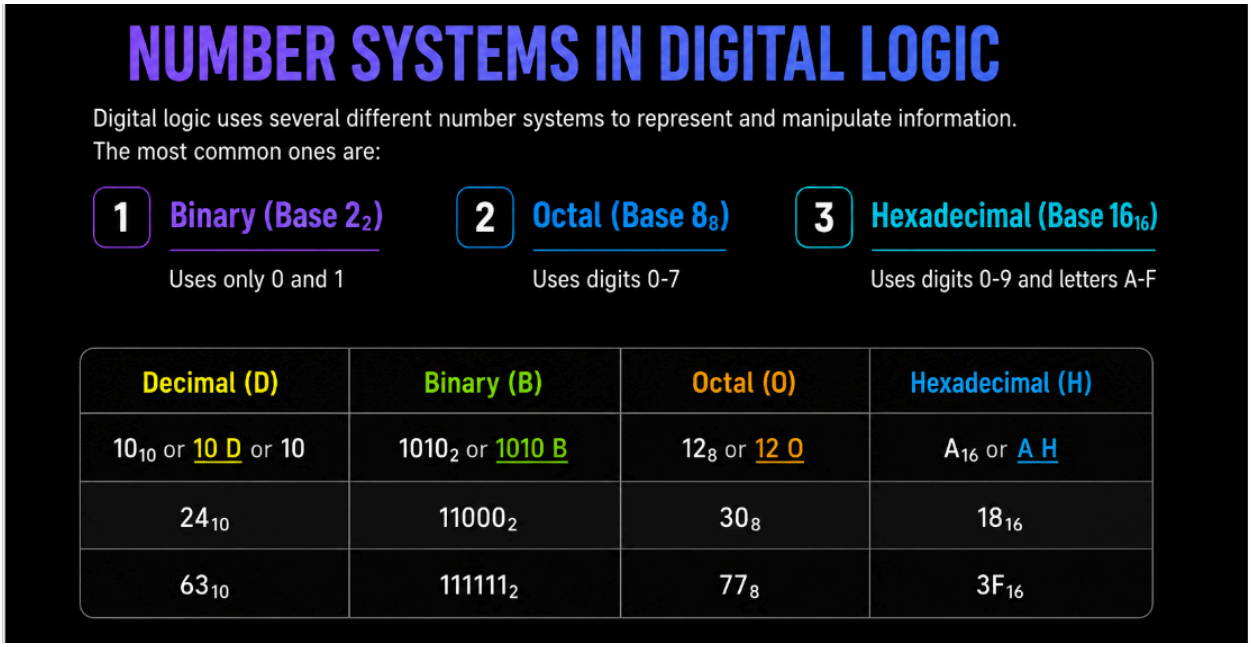
Midterm-test prepare / All theory

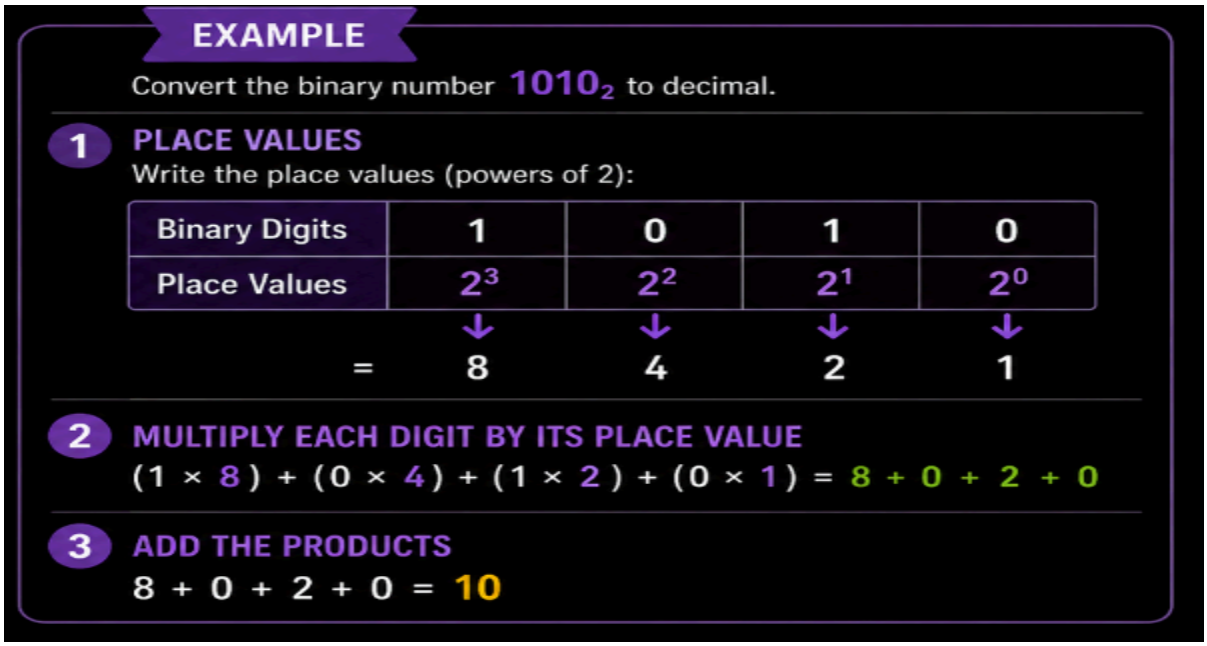
Chap 1



Number systems in digital logic - Working/Calculate step

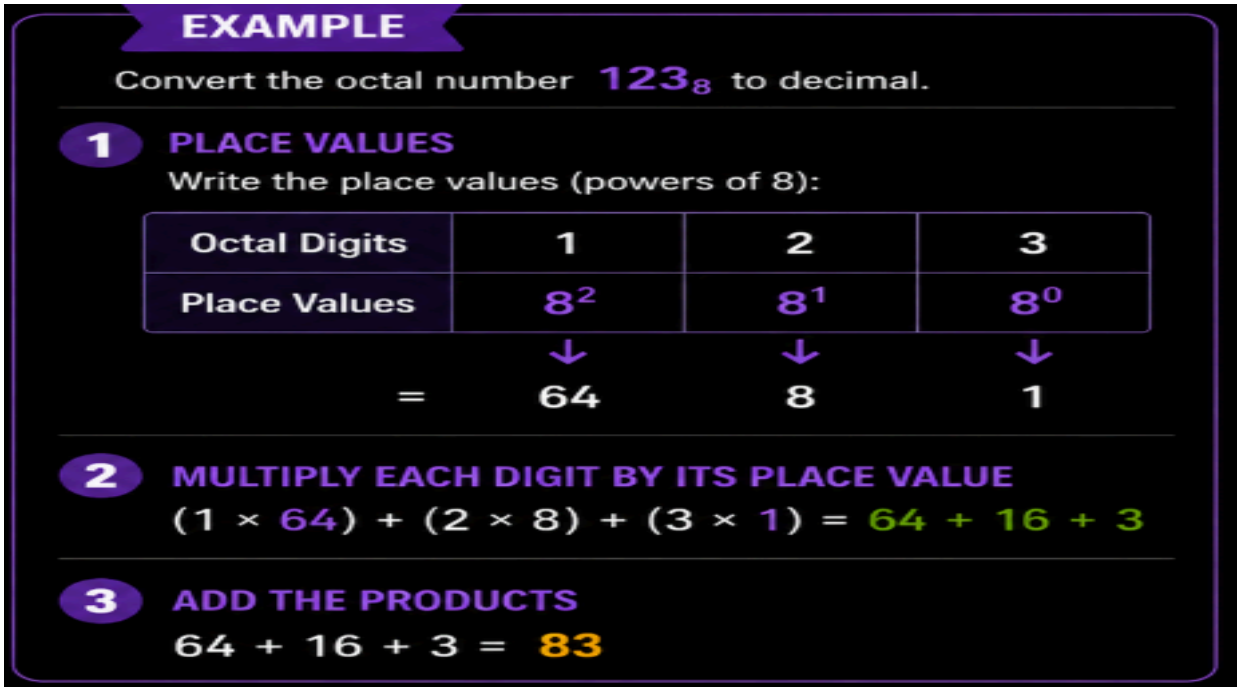
1.1 Any base to decimal - Converting Binary to Decimal

Write the binary number from right to left, with each digit's place value shown as powers of 2 , multiply each binary digit by its corresponding value, and sum all the products.



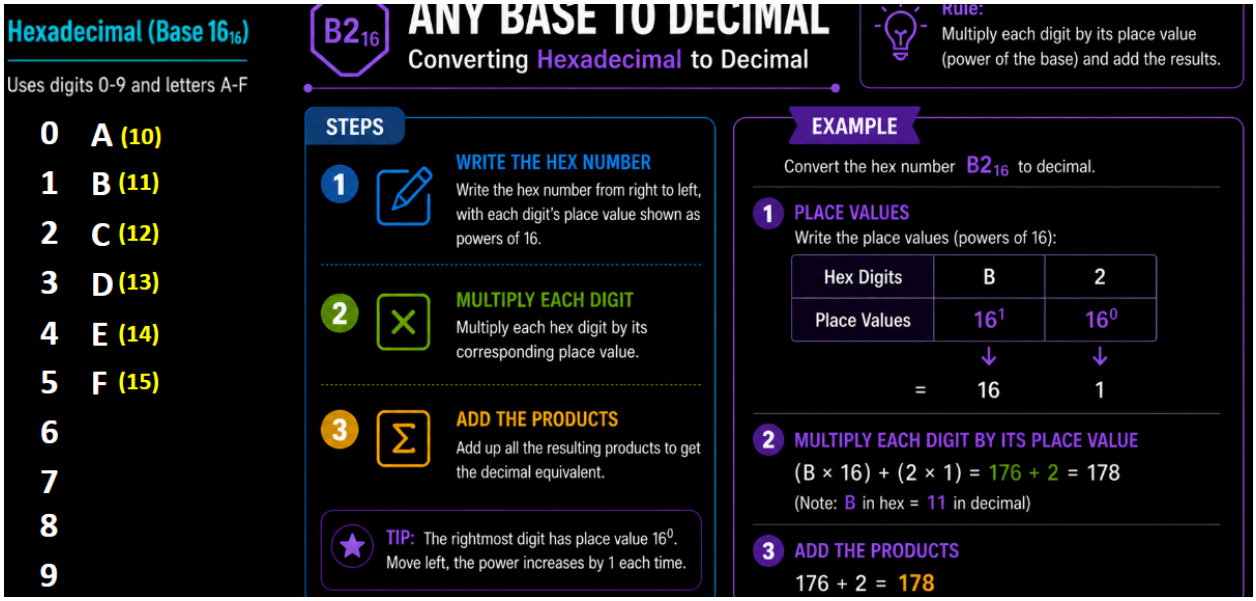
1.2 Any base to decimal - Converting Octal to Decimal

Write the Octal number from right to left, with each digit's place value shown as powers of 8 , multiply each octal digit by its corresponding value, and sum all the products.



1.3 Any base to decimal - Converting Hexadecimal to Decimal

Write the Hex number from right to left, with each digit's place value shown as powers of 16 , multiply each hex digit by its corresponding value, and sum all the products.



1.4 Any base to decimal - Maybe will ask : Perform the following conversions. You are required to show the working steps clearly. If the operation(s) is illogical, explain the reason.

Q : 397_8 to base-10 number

Illogical.

Because, base 8 only representing the digits from 0 - 7

Q : 346_5 to base-10 number

Illogical.

Because, base 5 only representing the digits from 0 - 4

Q : $HA4_{16}$ to base-10 number

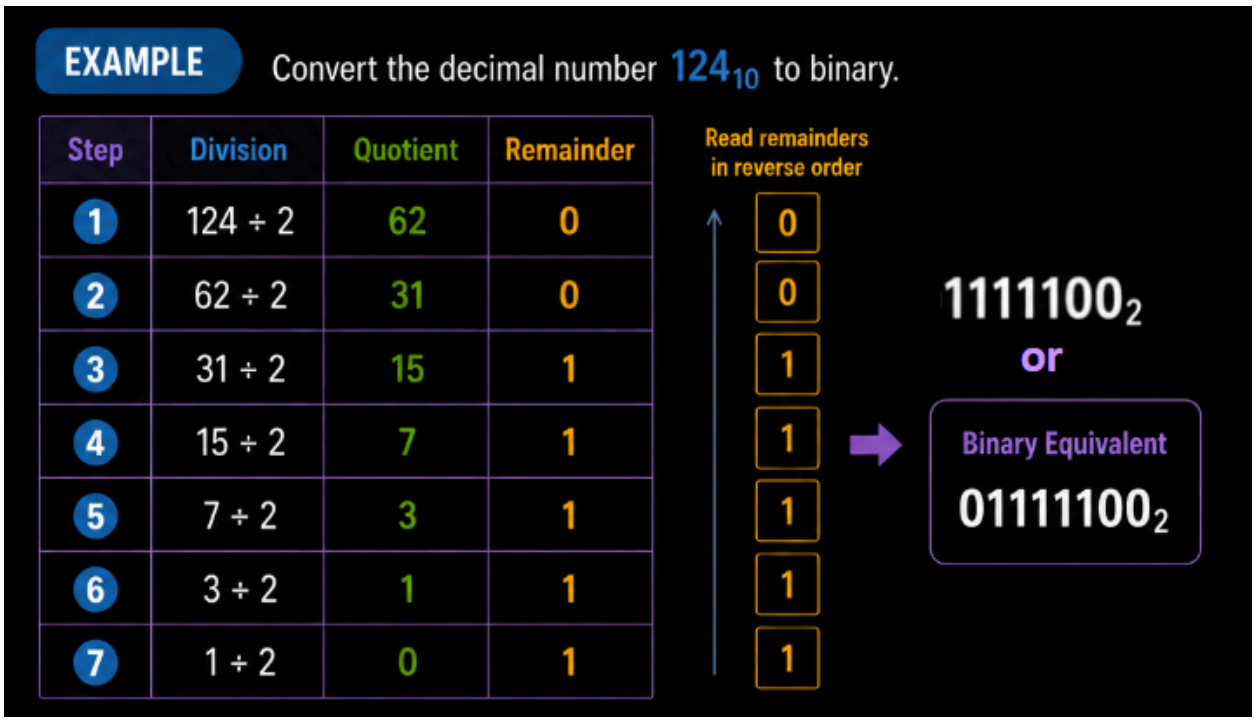
Illogical.

Because, base 16 only representing the digits from 0 – 9, A – F

Summary - Check the base, if number out of the area of the base is illogical

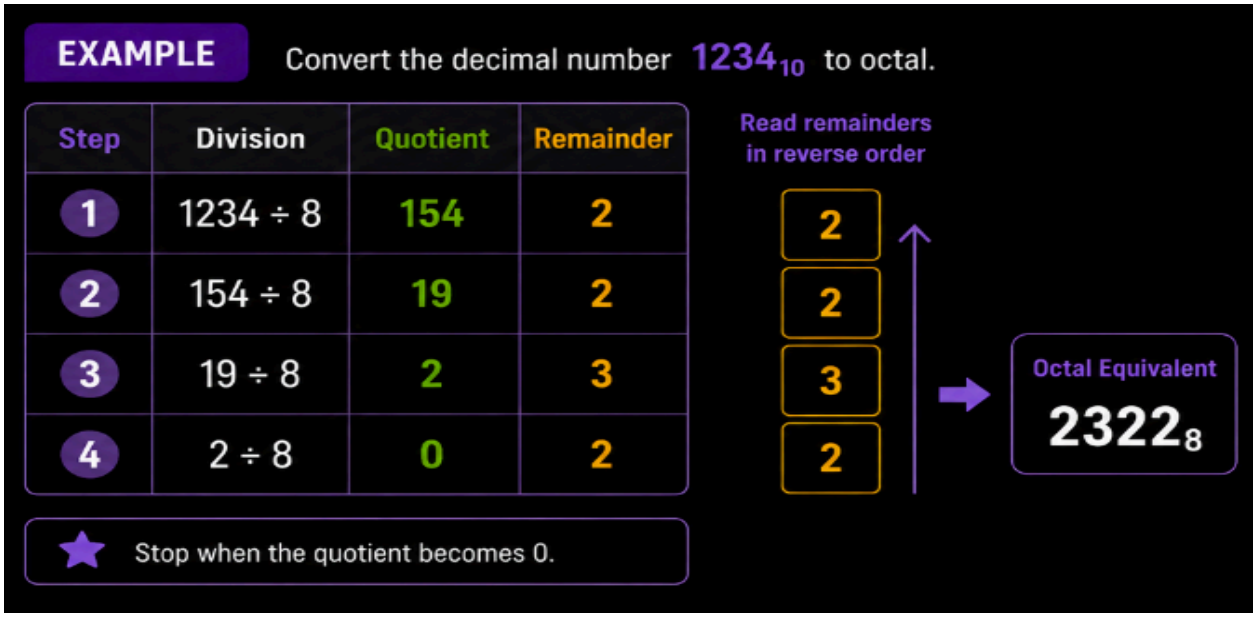
2.1 Decimal to any base - Converting Decimal to Binary

Repeatedly divide the decimal number by the 2, record the remainders at each step until the quotient is zero, and read them in reverse order.



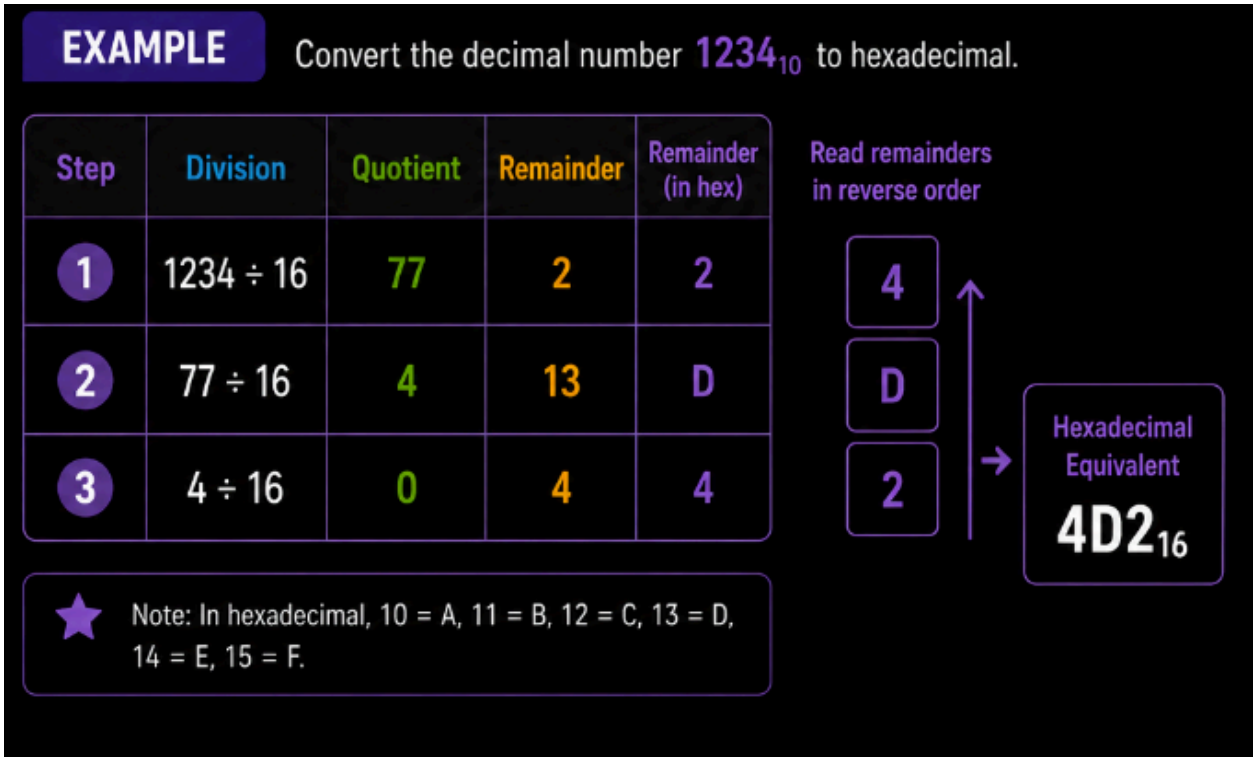
2.2 Decimal to any base - Converting Decimal to Octal

Repeatedly divide the decimal number by the 8, record the remainders at each step until the quotient is zero, and read them in reverse order.



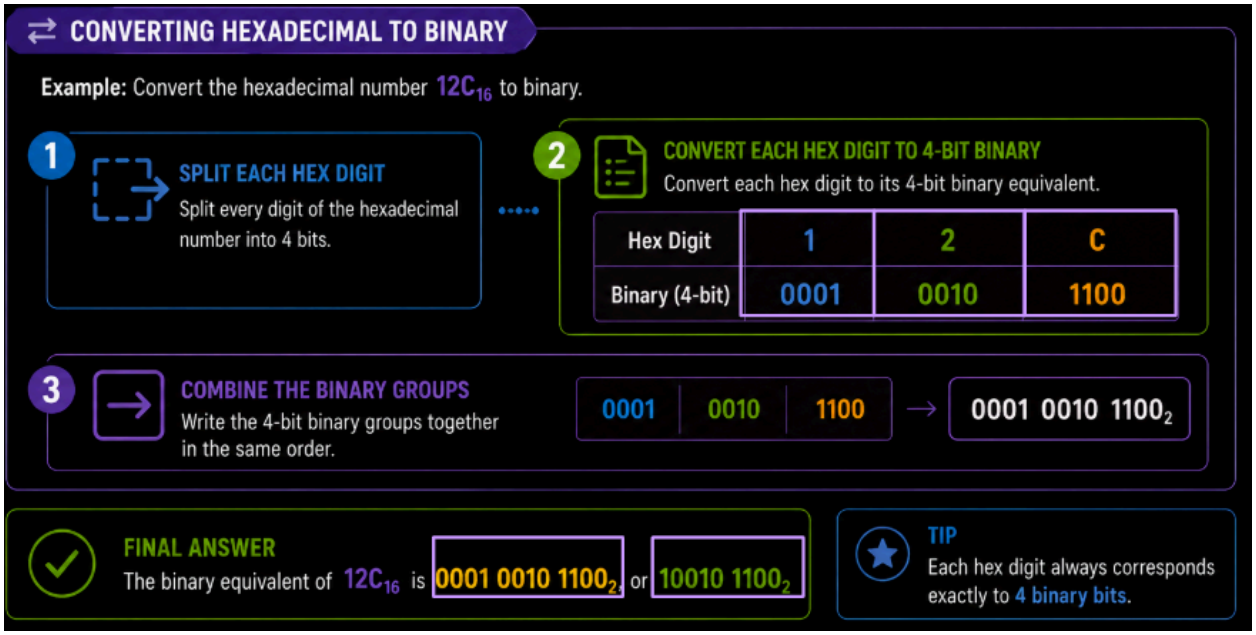
2.3 Decimal to any base - Converting Decimal to Hexadecimal

Repeatedly divide the decimal number by the 16, record the remainders at each step until the quotient is zero, and read them in reverse order.



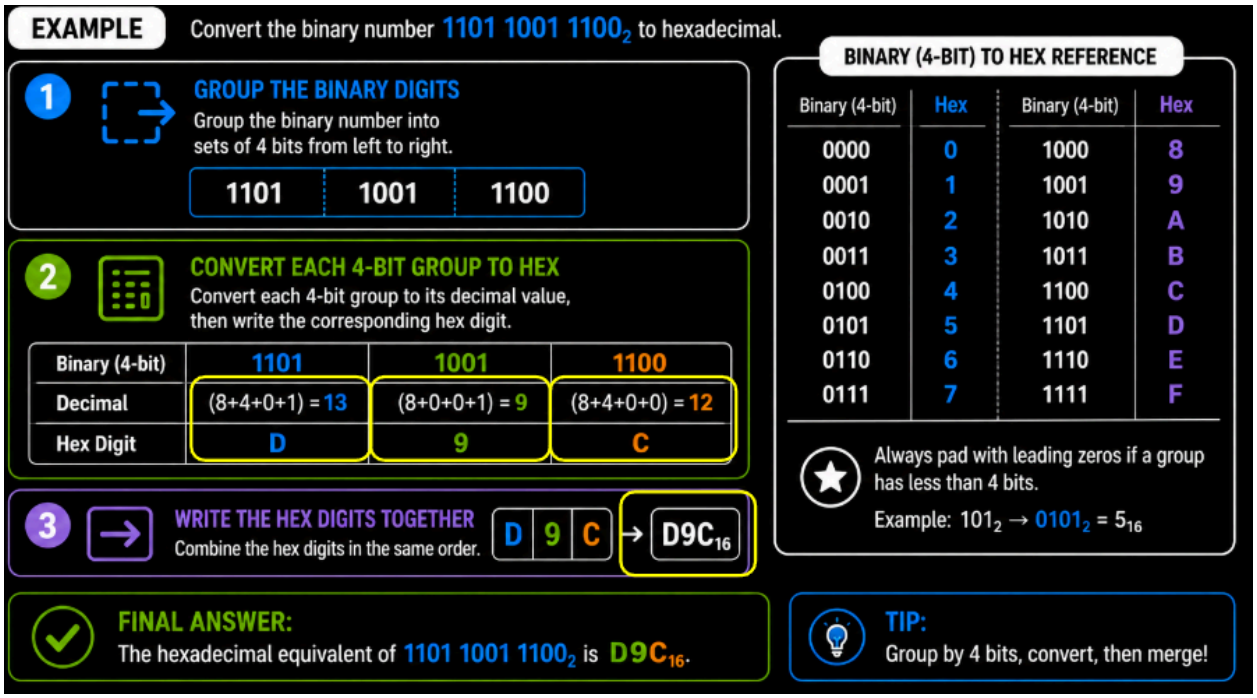
3.1 Another Conversions - Hexadecimal Conversions (splitting)

Convert each hexadecimal digit into its 4-bit binary equivalent and combine them together in the original order.



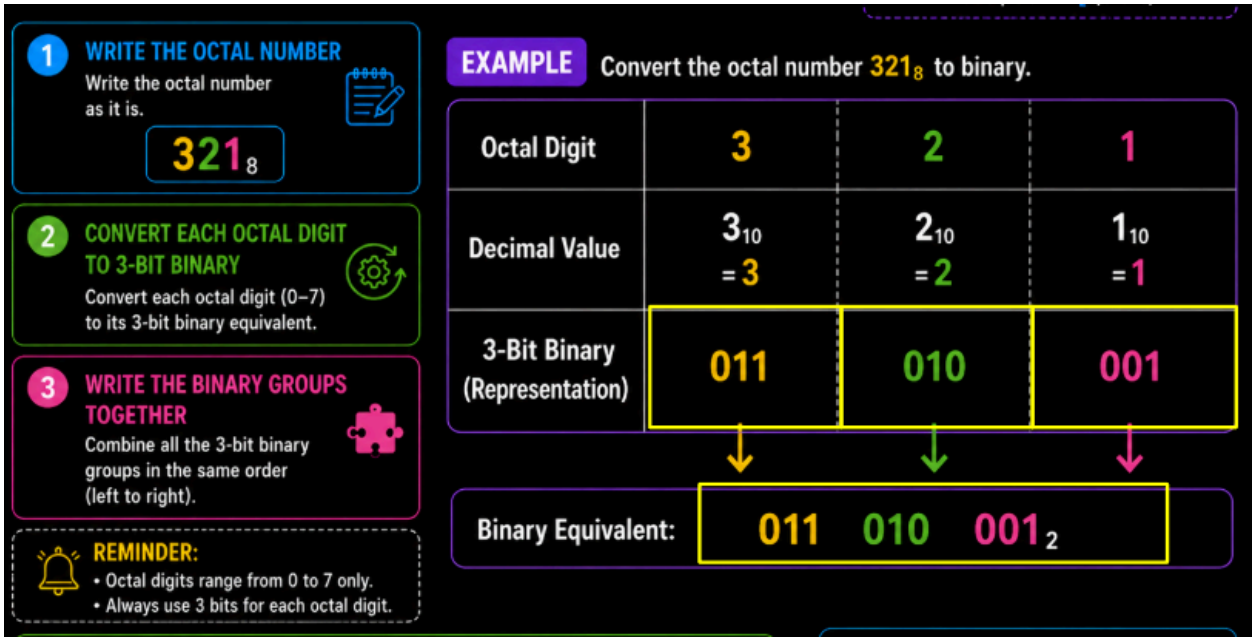
3.2 Another Conversions - Conversions Binary to Hexadecimal (merging)

Group the binary bits into fours from right to left, and convert each 4-bit group into its equivalent hexadecimal digit.



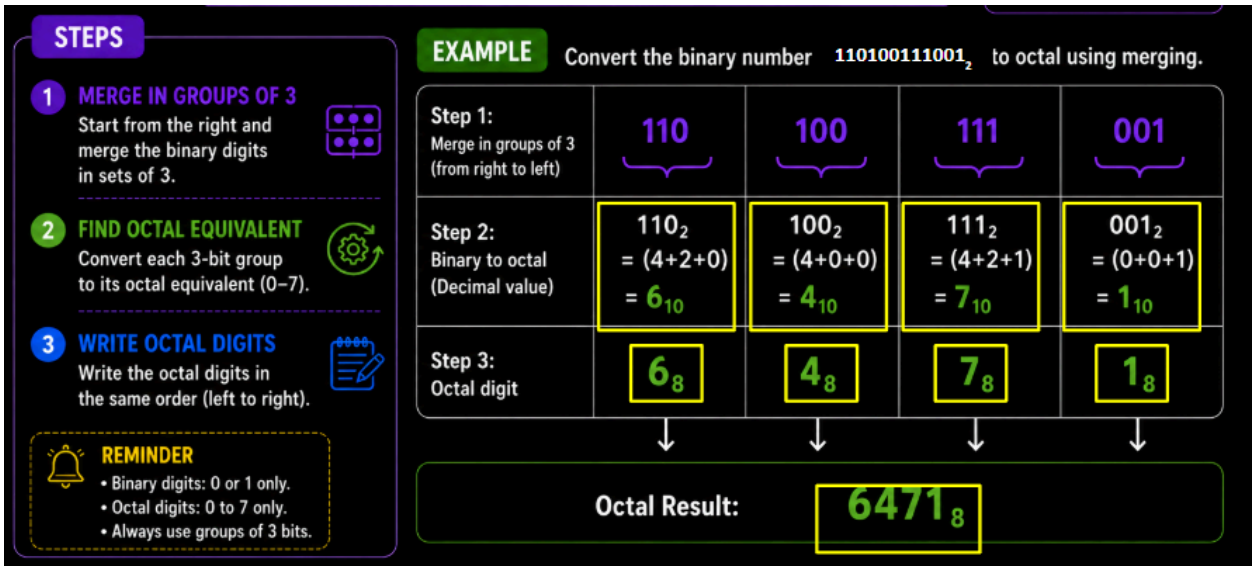
3.3 Another Conversions - Conversions Octal to Binary

Convert each octal digit into its 3-bit binary equivalent and combine them together in the original order.



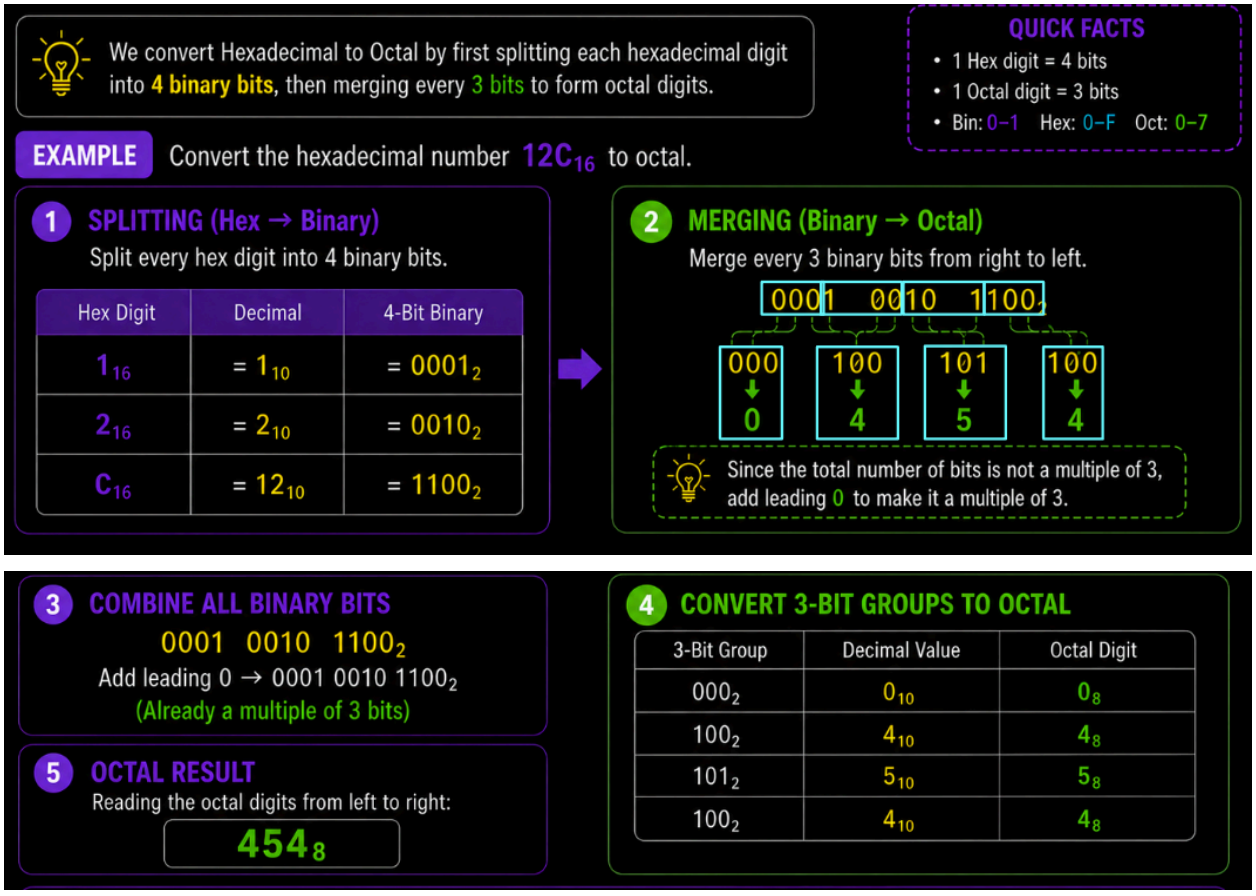
3.4 Another Conversions - Conversions Binary to Octal

Group the binary bits into threes from right to left, and convert each 3-bit group into its equivalent octal digit.



3.5 Another Conversions - Conversions Hexadecimal to Octal

Convert each hex digit into 4-bit binary, regroup the combined bits into threes from right to left, and convert each 3-bit group into its octal digit.



4. bitwise operations (explain)

AND (&): Returns 1 only if **both** input bits are 1; otherwise 0
OR (||): Returns 1 if **at least one** input bit is 1; returns 0 only when both bits are 0.
NOT (~): Inverts each bit. 0 becomes 1, 1 becomes 0.
XOR (): Returns 1 if input bits are **different**; returns 0 if inputs are same.

5. binary arithmetic ,decimal arithmetic, hexadecimal arithmetic

Page 20 - 34 自己读slides lecture slides page 20

Chap 2

1. Unsigned Binary & BCD (Page 2–9)

1.1 Unsigned Binary

- Represents numbers using the binary numeral system

Characteristics

- used only two digit: 0 and 1
- Direct binary equivalent and uses fewer bits

Advantages of Unsigned Binary

- faster processing
- less storage required

Examples : $\mathbf{68_{10} = 1000\ 0100_2}$

1.2 BCD (Binary-Coded Decimal)

- Each decimal digit is converted separately into 4-bit binary

Characteristics

- Common in calculators and displays
- Uses 4 bits per digit
- Easy conversion between decimal and BCD

Advantages of BCD

- Easier to read and interpret for humans
- Accurate financial calculations

Examples : $\mathbf{68_{10} = 0110\ 1000}$

2. SEEMMMMM FORMAT (Floating Point Representation)

Format Structure

- S EE MMMMM

S = Sign of mantissa (1 digit)
EE = Exponent (2 digits), stored using Excess (Bias) notation
MMMMM = Mantissa (5 digits), range $0 \leq \text{Mantissa} < 1$

STEP-BY-STEP PROCESS (Normalization & Encoding)

- Step 1: Normalize the number
Move decimal point so that mantissa is between 0 and 1
Example: 246.8035 = 0.2468035 × 10^3
Mantissa = 0.2468035
Actual Exponent = +3

- Step 2: Determine the Sign (S)
If number is positive → S = given positive digit (e.g. 0)
If number is negative → S = given negative digit (e.g. 9)

- Step 3: Determine the Exponent (EE) using Excess notation
Formula: Stored EE = Actual Exponent + Bias
Bias is specified in question (e.g. Excess-50, Excess-48, Excess-52)
Example: Actual Exponent = +3, Excess-50 → EE = 3 + 50 = 53

- Step 4: Determine the Mantissa (MMMMM)
Take the fractional part of normalized number
Remove decimal point
Take first 5 digits
Pad with zeros if needed

- Step 5: Assemble SEEMMMMM
Combine: S + EE + MMMMM
Example: 05324680

Decoding SEEMMMMM (Verification)

- Step 1: Extract S, EE, MMMMM
- Step 2: Determine sign from S
- Step 3: Calculate Actual Exponent
- Step 4: Reconstruct mantissa
- Step 5: Calculate final value

Chap 3
Register
1. Register (Flag register)
Flag Register Definition
- A special-purpose register in the CPU

Common Flags (Theory Points)
- Zero Flag (ZF) – Set when result of operation is zero
- Carry Flag (CF) – Set when an arithmetic operation generates a carry out of MSB
- Overflow Flag (OF) – Set when result exceeds the fixed bit range
- Sign Flag (SF) – Set when result is negative (MSB = 1)
- Parity Flag (PF) – Set when number of 1s in result is even

Purpose of Flag Register
- Used to indicate some special condition. (Exp : overflow, carry)
- Used by the CPU to make decisions (e.g., conditional jumps / branching)

2. Machine cycle
Definition
- The basic operation cycle of a CPU

4 Steps of Machine Cycle
- FETCH – Retrieve an instruction from memory
- DECODE – Translate the instruction into computer commands
- EXECUTE – Execute the commands
- STORE – Write the results back to memory

Fetch Phase
- PC (Program Counter) holds the address of the next instruction
- Contents at that memory address are read into the IR (Instruction Register)
- PC is incremented to point to the next instruction

Execution Phase – LOAD Instruction
- Copy address from instruction to MAR
- Copy contents of MDR into a general-purpose register

- Increment PC register

Execution Phase – ADD Instruction (Steps)
PC → MAR
MDR → IR
IR [address] → MAR
A + MDR → A
PC + 1 → PC

Execution Phase – STORE Instruction (Steps)
PC → MAR
MDR → IR
IR [address] → MAR
A → MDR
PC + 1 → PC

3. Buses
Bus Definition
- A physical connection that transfers data from one location to another

Three Separate Buses
Data Bus
- Carry the data being moved from one location to another
- Bidirectional bus

Address Bus
- Specifies the recipient of the data on the bus
- Unidirectional bus

Control Bus
- Provides control for proper synchronisation and operation
- Bidirectional bus

Point-to-Point Bus
- Carries signals from specific source to specific destination
- provides higher efficiency and speed
Example: cable connecting parallel or serial port to printer

Multipoint Bus
- Connects several points together
- Signals are broadcast to every other point on the bus
Example: Ethernet

4. Cripset

Chipset Definition
- A set of electronic components on the motherboard
- Manages data flow between the processor, memory, and peripherals

Two Main Components of a Chipset
North Bridge
South Bridge

North Bridge
- Also called Memory Controller Hub
- Responsible for controlling transfers between processor and RAM
- Handles high-speed communication

South Bridge
- Also called I/O Controller Hub
- Manages communications between peripheral devices and the processor
- Handles slower-speed communication

North Bridge vs South Bridge Summary

North Bridge	**South Bridge**
Memory controller hub	I/O controller hub
Connects to CPU and RAM	Connects to peripherals and I/O devices
Handles high-speed components	Handles slower-speed components

5. Special register (RISC V) (我直接**abandon**了，要的话自己读)